Numpy (Numerical Python)

```
import numpy as np
```

Creating Numpy Array and Studying its types

```
# normal array
list1=[2,4,6,8]
array = np.array(list1)
print(array)
```

```
[2 4 6 8]
```

```
# array of 0 and 1
array0 = np.zeros(4)
array1 = np.ones(4)
print(array0,array1)
```

```
[0. 0. 0. 0.] [1. 1. 1. 1.]
```

```
# creating array with range
arr1 = np.arange(5)
arr2 = np.arange(1,6)
arr3 = np.arange(1,11,2)
print(arr1,arr2,arr3)
```

```
[0 1 2 3 4] [1 2 3 4 5] [1 3 5 7 9]
```

```
# random array generator
arr = np.random.rand(5)
print(arr)
```

```
[0.83177336 0.10457029 0.03207396 0.96526122 0.61636389]
```

```
# random integer array generator
arr = np.random.randint(1,101,size=5)
print(arr)
```

```
[39 55 62 68 42]
```

```
# random integer array generator with seed
np.random.seed(0)
arr = np.random.randint(1,101,size=5)
print(arr)
```

```
[45 48 65 68 68]
```

```
# generate 5 random number uniformly distributed distributed between
0.0 and 1.0
```

```
arr = np.random.uniform(0.0,1.0,size=5)
print(arr)
```

```
[0.84725174 0.6235637  0.38438171 0.29753461 0.05671298]
```

```
arr = np.empty(5)
print(arr)
```

```
[0.84725174 0.6235637  0.38438171 0.29753461 0.05671298]
```

```
# 1d array
arr1d = np.zeros(5)
print(arr1d)
print(arr1d.shape)
```

```
[0. 0. 0. 0. 0.]
(5,)
```

```
# 2d array
arr2d = np.zeros((2,3))
print(arr2d)
print(arr2d.shape)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
(2, 3)
```

```
# 3d array
arr3d = np.zeros((2,3,4))
print(arr3d)
print(arr3d.shape)
```

```
[[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
(2, 3, 4)
```

```
# 4d array
arr4d = np.zeros((2,3,4,5))
print(arr4d)
print(arr4d.shape)
# (no of 3d arrays,no of 2d arrays,rows,columns)
```

```
[[[[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]
```

```
  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]]


 [[[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]

  [[0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0.]]]]
(2, 3, 4, 5)
```

```
arr = np.array([[1,2,3],[4,5,6]])
print(arr.ndim)
print(len(arr))
print(arr.shape)
print(arr.dtype.name)
print(arr.astype(int))
print(arr.dtype)
```

```
2
2
(2, 3)
int32
[[1 2 3]
 [4 5 6]]
int32
```

```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
print("Number of Dimension : ",arr.ndim)
print("Size of array : ",arr.size)
print("Shape of array : ",arr.shape)
print("Data type of elements : ",arr.dtype)
print("Size of each elements in bytes : ",arr.itemsize)
```

```
Number of Dimension :  2
Size of array :  9
Shape of array :  (3, 3)
Data type of elements :  int32
Size of each elements in bytes :  4
```

```
arr2 = np.array([[1,2,3],[4,5,6],[7,8,9]],dtype=np.int64)
print(arr2.itemsize)
print(arr2.data)
```

```
8
<memory at 0x0000024E90E92EA0>
```

```
arr2d = np.array([[1,2,3,4],[5,6,7,8]])
print(arr2d)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

```
arr4d = np.array([[[1,2,3,4],[5,6,7,8],[9,10,11,12]],[[1,2,3,4],
[5,6,7,8],[9,10,11,12]]])
print(arr4d)
```

```
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]]
```

```
arr = np.zeros((2,3))
print("2d array")
print(arr)
```

```
2d array
[[0. 0. 0.]
 [0. 0. 0.]]
```

```
arr = np.zeros((2,3,4))
print("3d array")
print(arr)
```

```
3d array
[[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
```

```
arr = np.ones((2,3,4))
print("3d array")
print(arr)
```

```
3d array
[[[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]]
```

```
numpy_array = np.full((2,2),5)
print(numpy_array)
```

```
[[5 5]
 [5 5]]
```

```
numpy_array_3d = np.random.rand(2,2,2)
print(numpy_array_3d)
```

```
[[[0.27265629 0.47766512]
  [0.81216873 0.47997717]]

 [[0.3927848  0.83607876]
  [0.33739616 0.64817187]]]
```

```
numpy_array_3d = np.empty((2,2,2))
print(numpy_array_3d)
```

```
[[[0.27265629 0.47766512]
  [0.81216873 0.47997717]]

 [[0.3927848  0.83607876]
  [0.33739616 0.64817187]]]
```

```
carr1 = np.array([1+2j,2+3j,3+8j])
print(carr1)
print(carr1.dtype)
print(carr1.itemsize)
```

```
[1.+2.j 2.+3.j 3.+8.j]
complex128
16
```

```
farr = carr1.astype('float')
print(farr)
```

```
[1. 2. 3.]
```

```
C:\Users\djsce.student\AppData\Local\Continuum\anaconda3\lib\site-packages\ipykernel_launcher.py:1: ComplexWarning: Casting complex values to real discards the imaginary part
  """Entry point for launching an IPython kernel.
```

Numpy array Indexing

starts with 0

```
arr1 = np.array([1,3,5,9])
print(arr1[0])
print(arr1[2])
print(arr1[-1])
```

```
1
5
9
```

```
arr1[1]=11
arr1[3]=13
print(arr1)
```

```
[ 1 11  5 13]
```

```
print(arr1[-2])
```

```
5
```

20th August

```
arr1 = np.array([[1, 3, 5], [7, 9, 2], [4, 6, 8]])
print("output : ",arr1[2, 1])
```

```
6
```

```
print("2nd Row: ",arr1[1,:])
```

```
2nd Row:  [7 9 2]
```

```
print("3rd column: ",arr1[:,2])
```

```
3rd column:  [5 2 8]
```

```
arr2 = np.array([[[1,2,3,4],[5,6,7,8],[9,10,11,12]],[[13,14,15,16],[17,18,19,20],[21,22,23,24]]])
print("element : ",arr2[1,2,1])
print("row : ",arr2[1,2,:])
print("column : ",arr2[1,:,1])
print("what to call it ? : ",arr2[:,2,1])
```

```
element :  22
row :  [21 22 23 24]
column :  [14 18 22]
what to call it ? :  [10 22]
```

```
arr = np.array([1,3,5,7,9,2,4,6,8])
print(arr[2:6])
print(arr[0:8:2])
print(arr[3:])
```

```
[5 7 9 2]
[1 5 9 4]
[7 9 2 4 6 8]
```

```
arr = np.array([10,20,30,40,50])
print(arr[:2])
print(arr[2:])
print(arr[::-1])
```

```
[10 20]
[30 40 50]
[50 40 30 20 10]
```

```
# changing by indexing
arr[3:]=100
print(arr)
```

```
[ 10  20  30 100 100]
```

```
arr[:3]=20
print(arr)
```

```
[ 20  20  20 100 100]
```

```
arr[::2]=0
print(arr)
```

```
[  0  20   0 100   0]
```

```
arr = np.array([10,20,30,40,50])
print(arr[-3:])
print(arr[-5:-2])
print(arr[-1::-2])
```

```
[30 40 50]
[10 20 30]
[50 30 10]
```

```
arr = np.array([[1,3,5,7],[9,11,13,15]])
print(arr[:2,:2])
```

```
[[ 1  3]
 [ 9 11]]

print(arr[:2,2:])

[[ 5  7]
 [13 15]]

arr = np.array([1,3,5,7,2,4,6,8])
result = np.reshape(arr,(2,4))
print(result)

[[1 3 5 7]
 [2 4 6 8]]

result = np.reshape(result,(2,2,2))
print(result)

[[[1 3]
  [5 7]]

 [[2 4]
  [6 8]]]

print(np.reshape(arr,(2,2,2)))

[[[1 3]
  [5 7]]

 [[2 4]
  [6 8]]]

# flattening or array
result = np.reshape(result,-1)
print(result)

[1 3 5 7 2 4 6 8]

# adding 2 arrays
arr1 = np.array([1,3,5,7])
arr2 = np.array([8,6,4,2])
print(arr1+arr2)

[9 9 9 9]

# mean,median,mode
marks = np.array([76,78,81,66,85])
print("mean : ",np.mean(marks))
print("median : ",np.median(marks))
print("minimum : ",np.min(marks))
print("maximum : ",np.max(marks))
```

```
mean :  77.2
median :  78.0
minimum :  66
maximum :  85
```

```
# less then , greater then , equal to
print("arr1 < arr2 : ",arr1 < arr2)
print("arr1 > arr2 : ",arr1 > arr2)
print("arr1 == arr2 : ",arr1 == arr2)
print("arr1 <= arr2 : ",arr1 <= arr2)
print("arr1 <= arr2 : ",np.less_equal(arr1,arr2))
print("arr1 >= arr2 : ",arr1 >= arr2)
```

```
arr1 < arr2 :  [ True  True False False]
arr1 > arr2 :  [False False  True  True]
arr1 == arr2 :  [False False False False]
arr1 <= arr2 :  [ True  True False False]
arr1 <= arr2 :  [ True  True False False]
arr1 >= arr2 :  [False False  True  True]
```

```
# column-wise axis=0
# row-wise axis=1 similar for mean,median,comparison
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(np.sum(arr,axis=0))
print(np.sum(arr,axis=1))
```

```
[12 15 18]
[ 6 15 24]
```

```
#slicing
arr=np.array([10,20,30,40,50,60,70,80])
print(arr[1:6])
```

```
[20 30 40 50 60]
```

```
#negative slicing
numbers=np.array([5,10,15,20,25,30,35,40])
print(numbers[-5:-1])
```

```
[20 25 30 35]
```

```
#slicing extract 2nd and 3d row
arr = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]])
print(arr[1:3,:])
```

```
[[ 5  6  7  8]
 [ 9 10 11 12]]
```

```
#slicing extract 1sr and 3d column
arr = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]])
print(arr[:,0:3:2])
```

```
[[ 1  3]
 [ 5  7]
 [ 9 11]
 [13 15]]
```

```
#numpy array 1 to 12 matrix 3x4
arr = np.arange(1,13)
arr = np.reshape(arr,(3,4))
print(arr)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

```
#numpy array 1 to 24 in 3d shape
arr = np.arange(1,25)
arr = np.reshape(arr,(2,3,4))
print(arr)
```

```
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]
```

```
#numpy array 1 to 20 in 4x5 and 5x4 shape
arr = np.arange(1,21)
arr = np.reshape(arr,(4,5))
print(arr)
arr = np.reshape(arr,(5,4))
print(arr)
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]]
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]]
```